

Python Libraries

Introduction

A Python Library is a collection of pre-written modules, functions, classes, and packages that provide reusable code to perform specific tasks. Libraries help developers avoid writing code from scratch and significantly speed up software development.

Python has a rich ecosystem of libraries for:

- General Programming
- Data Science
- Machine Learning
- Artificial Intelligence
- Web Development
- Cyber Security
- Digital Forensics
- Automation
- Scientific Computing

What is a Library?

A library is a collection of related modules that provide specialized functionality.

Example

```
import math  
  
print(math.sqrt(25))
```

Output:

```
5.0
```

Here, math is a Python library that provides mathematical functions.

Difference Between Module, Package, and Library

Feature	Module	Package	Library
Definition	Single Python file	Collection of modules	Collection of packages and modules

Example	math.py	numpy.linalg	NumPy
Scope	Small	Medium	Large
Purpose	Reusable code	Organize modules	Complete functionality

Types of Python Libraries

1. Standard Libraries

Libraries included with Python installation.

Examples:

```
math
random
os
sys
datetime
json
csv
re
statistics
```

2. Third-Party Libraries

Installed using PIP.

Examples:

```
NumPy
Pandas
Matplotlib
Scikit-Learn
TensorFlow
OpenCV
Requests
BeautifulSoup
```

Installing Libraries

Python uses PIP (Package Installer for Python).

Install a Library

```
pip install numpy
```

Install Multiple Libraries

```
pip install numpy pandas matplotlib
```

Check Installed Libraries

```
pip list
```

Upgrade a Library

```
pip install --upgrade numpy
```

Common Python Standard Libraries

1. Math Library

Provides mathematical functions.

Import

```
import math
```

Example

```
import math

print(math.sqrt(64))
print(math.factorial(5))
print(math.pi)
```

Output

```
8.0
120
3.141592653589793
```

Common Functions

Function	Purpose
sqrt()	Square root
factorial()	Factorial
ceil()	Round up
floor()	Round down
pow()	Power
pi	Pi constant

2. Random Library

Generates random numbers.

Example

```
import random  
  
print(random.randint(1,10))
```

Possible Output:

```
7
```

Common Functions

Function	Purpose
randint()	Random integer
random()	Random decimal
choice()	Random element
shuffle()	Shuffle list

3. Datetime Library

Handles dates and times.

Example

```
import datetime  
  
today = datetime.datetime.now()  
  
print(today)
```

Possible Output:

```
2026-06-01 12:30:45.123456
```

Common Functions

Function	Purpose
now()	Current date and time
date()	Current date
time()	Current time
timedelta()	Date calculations

4. OS Library

Provides operating system functions.

Example

```
import os

print(os.getcwd())
```

Output:

```
Current Working Directory
```

Common Functions

Function	Purpose
getcwd()	Current directory
makedirs()	Create folder
remove()	Delete file
rename()	Rename file
listdir()	List files

5. Sys Library

Provides system information.

Example

```
import sys

print(sys.version)
```

Output

```
Python Version Information
```

6. JSON Library

Used for JSON processing.

Example

```
import json

person = {
    "name": "John",
    "age": 25
}

data = json.dumps(person)

print(data)
```

Output:

```
{"name": "John", "age": 25}
```

7. CSV Library

Used to read and write CSV files.

Example

```
import csv

with open("students.csv","w",newline="") as file:

    writer = csv.writer(file)

    writer.writerow(["Name","Marks"])
```

8. Regular Expression (re) Library

Used for pattern matching.

Example

```
import re

text = "john@gmail.com"

pattern = r"\w+@\w+\.\w+"

result = re.match(pattern,text)

print(result)
```

Output

```
<re.Match object>
```

Applications:

```
Email Validation
Password Validation
Log Analysis
```

Popular Third-Party Libraries

1. NumPy

Introduction

NumPy (Numerical Python) is the fundamental library for numerical computing in Python. It provides powerful support for:

- Multi-dimensional arrays
- Mathematical operations
- Linear algebra
- Statistical analysis
- Random number generation
- Scientific computing
- Machine Learning
- Artificial Intelligence
- Data Science

NumPy is one of the most widely used Python libraries and serves as the foundation for libraries such as:

- Pandas
- Scikit-learn
- TensorFlow
- PyTorch
- OpenCV

Why NumPy?

Before NumPy:

```
numbers = [10, 20, 30, 40]

result = []

for i in numbers:
    result.append(i * 2)

print(result)
```

Output:

```
[20, 40, 60, 80]
```

Using NumPy:

```
import numpy as np

numbers = np.array([10, 20, 30, 40])

print(numbers * 2)
```

Output:

```
[20 40 60 80]
```

Advantages:

- Faster execution
- Less memory consumption
- Easy mathematical operations
- Optimized C implementation

Installation

Install NumPy using PIP:

```
pip install numpy
```

Check installation:

```
import numpy as np

print(np.__version__)
```

Importing NumPy

```
import numpy as np
```

np is the standard alias used worldwide.

What is ndarray?

The core data structure in NumPy is called ndarray (N-Dimensional Array).

Definition:

An ndarray is a homogeneous collection of elements stored in contiguous memory locations.

Example:

```
import numpy as np

arr = np.array([10, 20, 30, 40])
```

```
print(arr)
```

Output:

```
[10 20 30 40]
```

Creating NumPy Arrays

From List

```
import numpy as np  
  
arr = np.array([1, 2, 3, 4])  
  
print(arr)
```

Output:

```
[1 2 3 4]
```

From Tuple

```
arr = np.array((10, 20, 30))
```

Specify Data Type

```
arr = np.array([1, 2, 3], dtype=float)  
  
print(arr)
```

Output:

```
[1. 2. 3.]
```

Array Dimensions

0-D Array (Scalar)

```
arr = np.array(100)  
  
print(arr)
```

Output:

```
100
```

1-D Array

```
arr = np.array([1,2,3,4])
```

Output:

```
[1 2 3 4]
```

2-D Array

```
arr = np.array([
    [1,2,3],
    [4,5,6]
])
print(arr)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

3-D Array

```
arr = np.array([
    [[1,2],[3,4]],
    [[5,6],[7,8]]
])
print(arr)
```

Output:

```
[[[1 2]
  [3 4]]

 [[5 6]
  [7 8]]]
```

Array Properties

```
ndim
```

Number of dimensions.

```
arr = np.array([[1,2],[3,4]])
print(arr.ndim)
```

Output:

```
2
```

shape

Displays dimensions.

```
print(arr.shape)
```

Output:

```
(2, 2)
```

Meaning:

```
2 Rows  
2 Columns
```

size

Number of elements.

```
print(arr.size)
```

Output:

```
4
```

dtype

Data type.

```
print(arr.dtype)
```

Output:

```
int64
```

Special Array Creation Functions

```
zeros()  
  
arr = np.zeros((3,3))  
  
print(arr)
```

Output:

```
[[0. 0. 0.]  
 [0. 0. 0.]  
 [0. 0. 0.]]
```

```
ones()  
arr = np.ones((2,3))
```

Output:

```
[[1. 1. 1.]  
 [1. 1. 1.]]
```

full()

```
arr = np.full((2,2), 7)  
print(arr)
```

Output:

```
[[7 7]  
 [7 7]]
```

eye()

Identity Matrix

```
arr = np.eye(3)  
print(arr)
```

Output:

```
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

Range Functions

```
arange()  
arr = np.arange(1,11)  
print(arr)
```

Output:

```
[1 2 3 4 5 6 7 8 9 10]
```

linspace()

Creates evenly spaced numbers.

```
arr = np.linspace(0,10,5)  
  
print(arr)
```

Output:

```
[ 0.  2.5  5.  7.5 10.]
```

Array Indexing

```
arr = np.array([10,20,30,40])  
  
print(arr[0])
```

Output:

```
10
```

Negative Indexing

```
print(arr[-1])
```

Output:

```
40
```

Array Slicing

```
arr = np.array([10,20,30,40,50])  
  
print(arr[1:4])
```

Output:

```
[20 30 40]
```

Mathematical Operations

```
a = np.array([1,2,3])  
b = np.array([4,5,6])  
  
print(a+b)
```

Output:

```
[5 7 9]
```

Subtraction

```
print(a-b)
```

Output:

```
[-3 -3 -3]
```

Multiplication

```
print(a*b)
```

Output:

```
[ 4 10 18]
```

Division

```
print(a/b)
```

Output:

```
[0.25 0.4 0.5]
```

Statistical Functions

```
arr = np.array([10,20,30,40,50])
```

Mean

```
print(np.mean(arr))
```

Output:

```
30.0
```

Median

```
print(np.median(arr))
```

Output:

```
30.0
```

Standard Deviation

```
print(np.std(arr))
```

Output:

```
14.14213562
```

Variance

```
print(np.var(arr))
```

Output:

```
200.0
```

Reshaping Arrays

```
arr = np.arange(12)

new_arr = arr.reshape(3,4)

print(new_arr)
```

Output:

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

Flattening Arrays

```
arr = np.array([
    [1,2],
    [3,4]
])

print(arr.flatten())
```

Output:

```
[1 2 3 4]
```

Random Numbers

```
import numpy as np

print(np.random.rand(3))
```

Output:

```
[0.23 0.75 0.42]
```

(Random values vary.)

Matrix Operations

Matrix Addition

```
A = np.array([
    [1,2],
    [3,4]
])

B = np.array([
    [5,6],
    [7,8]
])

print(A+B)
```

Output:

```
[[ 6  8]
 [10 12]]
```

Matrix Multiplication

```
print(np.dot(A,B))
```

Output:

```
[[19 22]
 [43 50]]
```

Linear Algebra

Determinant

```
A = np.array([
    [1,2],
    [3,4]
])

print(np.linalg.det(A))
```

Output:

```
-2.0
```

Inverse

```
print(np.linalg.inv(A))
```

Broadcasting

Broadcasting allows operations on arrays of different sizes.

```
arr = np.array([1,2,3])  
  
print(arr + 10)
```

Output:

```
[11 12 13]
```

NumPy in Data Science

Example Dataset:

```
import numpy as np  
  
marks = np.array([  
    85,90,78,88,92  
)  
  
print(np.mean(marks))
```

Output:

```
86.6
```

Applications:

- Data Cleaning
- Feature Engineering
- Statistical Analysis

NumPy in Machine Learning

```
features = np.array([  
    [1,2],  
    [3,4],  
    [5,6]  
)  
  
labels = np.array([
```

```
0,1,1  
)
```

Used for:

- Model Training
- Feature Matrices
- Tensor Operations

NumPy in Artificial Intelligence

Applications:

- Neural Networks
- Deep Learning
- Computer Vision
- Natural Language Processing

Example:

```
weights = np.random.rand(3,3)  
bias = np.random.rand(3)
```

NumPy in Cyber Security

Log Analysis

```
packet_sizes = np.array([  
    120,  
    350,  
    450,  
    200  
)  
  
print(np.mean(packet_sizes))
```

Traffic Analysis

```
network_data = np.array([  
    500,  
    700,  
    650,  
    900  
)
```

Advantages of NumPy

1. High Performance
2. Memory Efficient
3. Multi-dimensional Arrays
4. Powerful Mathematical Functions
5. Linear Algebra Support
6. Statistical Analysis Tools
7. Foundation of Data Science and AI

Limitations of NumPy

1. Stores homogeneous data only.
2. Less suitable for highly heterogeneous data.
3. Requires additional libraries for advanced data analysis (e.g., Pandas).

Common Interview Questions

Difference Between List and NumPy Array

Feature	List	NumPy Array
Speed	Slower	Faster
Memory	More	Less
Mathematical Operations	Limited	Extensive
Dimensions	Flexible	Multi-dimensional
Data Type	Mixed	Homogeneous

Practical Experiment: Student Marks Analysis

```
import numpy as np

marks = np.array([
    85,90,78,88,92
])

print("Average:", np.mean(marks))
print("Highest:", np.max(marks))
print("Lowest:", np.min(marks))
print("Standard Deviation:", np.std(marks))
```

Output

Average: 86.6
Highest: 92
Lowest: 78
Standard Deviation: 4.96

Summary

NumPy (Numerical Python) is the core scientific computing library in Python. It provides the powerful ndarray data structure, high-speed numerical computations, matrix operations, statistical functions, linear algebra capabilities, random number generation, and broadcasting. NumPy serves as the foundation for Data Science, Machine Learning, Artificial Intelligence, Computer Vision, Scientific Research, Cyber Security Analytics, and High-Performance Computing applications.

2. Pandas

Introduction

Pandas is an open-source Python library used for **data manipulation, data analysis, data cleaning, and data processing**. It provides powerful and flexible data structures that make working with structured data easy and efficient.

Pandas is built on top of **NumPy** and is one of the most widely used libraries in:

- Data Science
- Machine Learning
- Artificial Intelligence
- Business Analytics
- Financial Analysis
- Cyber Security Analytics
- Digital Forensics
- Research and Statistics

Why Pandas?

Consider student data:

Name	Marks	Age
John	85	20
Alice	92	21

David	78	19
--------------	----	----

Without Pandas, handling such data is difficult.

With Pandas:

```
import pandas as pd

data = {
    "Name":["John","Alice","David"],
    "Marks":[85,92,78],
    "Age":[20,21,19]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
   Name  Marks  Age
0  John     85   20
1  Alice    92   21
2  David    78   19
```

Features of Pandas

1. Fast and efficient data manipulation.
2. Easy handling of missing values.
3. Powerful data filtering and grouping.
4. Support for CSV, Excel, JSON, SQL, XML.
5. Time-series analysis support.
6. Statistical operations.
7. Data cleaning and preprocessing.
8. Integration with NumPy, Matplotlib, and Scikit-Learn.

Installation

Install Pandas using PIP:

```
pip install pandas
```

Check installation:

```
import pandas as pd

print(pd.__version__)
```

Importing Pandas

```
import pandas as pd
```

pd is the standard alias used worldwide.

Core Data Structures in Pandas

Pandas provides two primary data structures:

1. Series (1-Dimensional)
2. DataFrame (2-Dimensional)

Pandas Series

Definition

A **Series** is a one-dimensional labeled array capable of storing any data type.

Creating a Series

```
import pandas as pd

s = pd.Series([10,20,30,40])

print(s)
```

Output:

```
0    10
1    20
2    30
3    40
dtype: int64
```

Custom Index

```
s = pd.Series(
    [85,90,78],
    index=["John","Alice","David"]
)

print(s)
```

Output:

```
John    85
Alice   90
David   78
dtype: int64
```

Accessing Series Elements

```
print(s["John"])
```

Output:

```
85
```

Pandas DataFrame

Definition

A **DataFrame** is a two-dimensional tabular data structure consisting of rows and columns.

It is similar to:

- Excel Spreadsheet
- Database Table
- CSV File

Creating DataFrame from Dictionary

```
import pandas as pd

data = {
    "Name":["John","Alice","David"],
    "Marks":[85,92,78],
    "Age":[20,21,19]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
   Name  Marks  Age
0  John     85   20
1  Alice    92   21
2  David    78   19
```

DataFrame Attributes

shape

Returns rows and columns.

```
print(df.shape)
```

Output:

```
(3, 3)
```

Meaning:

```
3 Rows  
3 Columns
```

columns

```
print(df.columns)
```

Output:

```
Index(['Name', 'Marks', 'Age'], dtype='object')
```

index

```
print(df.index)
```

Output:

```
RangeIndex(start=0, stop=3, step=1)
```

dtypes

```
print(df.dtypes)
```

Output:

```
Name    object  
Marks   int64  
Age     int64
```

Viewing Data

head()

Displays first rows.

```
print(df.head())
```

head(2)

```
print(df.head(2))
```

Output:

	Name	Marks	Age
0	John	85	20
1	Alice	92	21

tail()

Displays last rows.

```
print(df.tail())
```

Selecting Columns

```
print(df["Name"])
```

Output:

0	John
1	Alice
2	David

Multiple Columns

```
print(df[["Name","Marks"]])
```

Output:

	Name	Marks
0	John	85
1	Alice	92
2	David	78

Selecting Rows

Using loc[]

Label-based selection.

```
print(df.loc[0])
```

Output:

Name	John
Marks	85
Age	20

Multiple Rows

```
print(df.loc[0:1])
```

Using iloc[]

Position-based selection.

```
print(df.iloc[1])
```

Output:

Name	Alice
Marks	92
Age	21

Adding Columns

```
df["Grade"] = ["A","A+","B"]
```

```
print(df)
```

Output:

	Name	Marks	Age	Grade
0	John	85	20	A
1	Alice	92	21	A+
2	David	78	19	B

Removing Columns

```
df.drop("Grade", axis=1)
```

Removing Rows

```
df.drop(0)
```

Removes first row.

Filtering Data

Students Scoring Above 80

```
print(df[df["Marks"] > 80])
```

Output:

	Name	Marks	Age
0	John	85	20
1	Alice	92	21

Sorting Data

Ascending Order

```
df.sort_values("Marks")
```

Descending Order

```
df.sort_values(  
    "Marks",  
    ascending=False  
)
```

Statistical Functions

```
print(df["Marks"].mean())
```

Output:

```
85.0
```

Maximum

```
print(df["Marks"].max())
```

Output:

```
92
```

Minimum

```
print(df["Marks"].min())
```

Output:

```
78
```

Sum

```
print(df["Marks"].sum())
```

Output:

```
255
```

Standard Deviation

```
print(df["Marks"].std())
```

Descriptive Statistics

```
print(df.describe())
```

Output:

	Marks	Age
count	3.000000	3.000000
mean	85.000000	20.000000
std	7.000000	1.000000
min	78.000000	19.000000
max	92.000000	21.000000

Handling Missing Values

Detect Missing Values

```
print(df.isnull())
```

Count Missing Values

```
print(df.isnull().sum())
```

Fill Missing Values

```
df.fillna(0)
```

Remove Missing Values

```
df.dropna()
```

Reading CSV Files

Sample CSV

```
Name,Marks,Age  
John,85,20  
Alice,92,21  
David,78,19
```

Read CSV

```
df = pd.read_csv("students.csv")  
  
print(df)
```

Writing CSV

```
df.to_csv(  
    "output.csv",  
    index=False  
)
```

Reading Excel Files

```
df = pd.read_excel(  
    "students.xlsx"  
)
```

Writing Excel Files

```
df.to_excel(  
    "output.xlsx",  
    index=False  
)
```

Reading JSON

```
df = pd.read_json(  
    "students.json"  
)
```

Writing JSON

```
df.to_json("output.json")
```

GroupBy Operation

```
data = {
    "Department":
        ["IT","HR","IT","HR"],
    "Salary":
        [50000,40000,60000,45000]
}

df = pd.DataFrame(data)

print(
    df.groupby(
        "Department"
    )["Salary"].mean()
)
```

Output:

```
Department
HR    42500
IT    55000
```

Merging DataFrames

```
df1 = pd.DataFrame({
    "ID":[1,2],
    "Name":["John","Alice"]
})

df2 = pd.DataFrame({
    "ID":[1,2],
    "Marks":[85,92]
})

result = pd.merge(
    df1,
    df2,
    on="ID"
)

print(result)
```

Output:

```
ID  Name  Marks
0  1  John    85
1  2  Alice   92
```

Time Series Analysis

```
dates = pd.date_range(  
    "2026-01-01",  
    periods=5  
)  
  
print(dates)
```

Output:

```
DatetimeIndex([  
'2026-01-01',  
'2026-01-02',  
'2026-01-03',  
'2026-01-04',  
'2026-01-05'  
)
```

Pandas in Data Science

```
import pandas as pd  
  
df = pd.read_csv(  
    "dataset.csv"  
)  
  
print(df.describe())
```

Uses:

- Data Cleaning
- Feature Engineering
- Statistical Analysis

Pandas in Machine Learning

```
df = pd.read_csv(  
    "training_data.csv"  
)  
  
X = df.drop(  
    "Target",  
    axis=1  
)  
  
y = df["Target"]
```

Used for:

- Data Preprocessing

- Data Transformation
- Feature Selection

Pandas in Cyber Security

Log Analysis

```
logs = pd.read_csv(
    "security_logs.csv"
)

failed = logs[
    logs["Status"] == "Failed"
]

print(failed)
```

Pandas in Digital Forensics

Timeline Analysis

```
evidence = pd.read_csv(
    "timeline.csv"
)

print(
    evidence.sort_values(
        "Timestamp"
    )
)
```

Important Pandas Functions

Function	Purpose
DataFrame()	Create DataFrame
Series()	Create Series
read_csv()	Read CSV
to_csv()	Write CSV
read_excel()	Read Excel
head()	First rows
tail()	Last rows

describe()	Statistics
info()	Dataset information
sort_values()	Sorting
groupby()	Grouping
merge()	Joining tables
fillna()	Fill missing values
dropna()	Remove missing values

Advantages of Pandas

1. Easy data manipulation.
2. Efficient handling of large datasets.
3. Powerful filtering and grouping.
4. Supports multiple file formats.
5. Excellent integration with NumPy and Machine Learning libraries.
6. Essential for Data Science and AI projects.

Limitations of Pandas

1. Memory-intensive for extremely large datasets.
2. Slower than specialized big-data frameworks for terabyte-scale data.
3. Not ideal for real-time streaming analytics.

Practical Experiment: Student Performance Analysis

```
import pandas as pd

data = {
    "Name":
        ["John","Alice","David"],
    "Marks":
        [85,92,78]
}

df = pd.DataFrame(data)

print("Average Marks:",
      df["Marks"].mean())
```

```
print("Highest Marks:",  
      df["Marks"].max())  
  
print("Lowest Marks:",  
      df["Marks"].min())
```

Output

```
Average Marks: 85.0  
Highest Marks: 92  
Lowest Marks: 78
```

Summary

Pandas is the most important Python library for **data analysis and manipulation**. It provides two powerful data structures—**Series** and **DataFrame**—for handling structured data efficiently. Pandas supports data cleaning, filtering, grouping, aggregation, merging, statistical analysis, and file handling (CSV, Excel, JSON, SQL). It is a core technology in Data Science, Machine Learning, Artificial Intelligence, Cyber Security Analytics, Financial Analytics, and Research Computing.

3. Matplotlib

Introduction

Matplotlib is one of the most popular Python libraries used for **data visualization**. It helps transform raw numerical data into meaningful graphical representations such as charts, graphs, plots, and diagrams.

Matplotlib is widely used in:

- Data Science
- Machine Learning
- Artificial Intelligence
- Business Analytics
- Scientific Research
- Cyber Security Analytics
- Digital Forensics
- Academic Research

The library was created by John D. Hunter in 2003 and has become the standard visualization library in Python.

Why Use Matplotlib?

Consider student marks:

```
marks = [85, 90, 78, 95, 88]
```

Viewing numbers directly:

```
85 90 78 95 88
```

is difficult to analyze.

Using Matplotlib:



Patterns become easier to understand.

Features of Matplotlib

- Easy to use
- High-quality visualizations
- Supports multiple chart types
- Works with NumPy and Pandas
- Highly customizable
- Publication-quality figures
- Interactive plotting support

Installation

Install using PIP:

```
pip install matplotlib
```

Verify installation:

```
import matplotlib  
  
print(matplotlib.__version__)
```

Importing Matplotlib

The plotting module is called **pyplot**.

```
import matplotlib.pyplot as plt  
plt is the standard alias.
```

Basic Plot

```
import matplotlib.pyplot as plt  
  
x = [1,2,3,4,5]  
y = [10,20,30,40,50]  
  
plt.plot(x, y)  
  
plt.show()
```

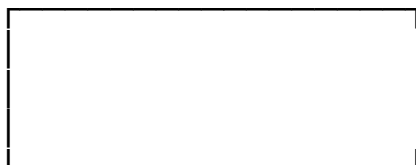
Output

A line graph connecting the points:

```
(1,10)  
(2,20)  
(3,30)  
(4,40)  
(5,50)
```

Components of a Graph

Title



Graph



X-Axis

Y-Axis



Adding Title and Labels

```
import matplotlib.pyplot as plt  
  
x = [1,2,3,4]  
y = [10,20,30,40]  
  
plt.plot(x,y)
```

```
plt.title("Student Performance")
plt.xlabel("Semester")
plt.ylabel("Marks")

plt.show()
```

Line Plot

A line plot shows trends over time.

```
import matplotlib.pyplot as plt

months = [1,2,3,4,5]
sales = [100,150,200,180,250]

plt.plot(months, sales)

plt.show()
```

Applications:

- Stock Prices
- Weather Trends
- Network Traffic Monitoring
- Sales Analysis

Customizing Line Plots

```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y = [10,20,30,40]

plt.plot(
    x,
    y,
    linestyle="--",
    marker="o",
    linewidth=2
)

plt.show()
```

Marker Types

Marker	Description
o	Circle
*	Star
s	Square
^	Triangle
+	Plus
x	Cross

Example:

```
plt.plot(x,y,marker="o")
```

Bar Chart

Used for categorical comparisons.

```
import matplotlib.pyplot as plt

students = ["John","Alice","David"]

marks = [85,92,78]

plt.bar(students, marks)

plt.show()
```

Applications:

- Student Performance
- Sales Comparison
- Attack Statistics
- Employee Analysis

Horizontal Bar Chart

```
plt.barh(students, marks)

plt.show()
```

Pie Chart

Shows percentage distribution.

```
import matplotlib.pyplot as plt

sizes = [40,30,20,10]

labels = [
    "Python",
    "Java",
    "C++",
    "JavaScript"
]

plt.pie(
    sizes,
    labels=labels,
    autopct="%1.1f%%"
)

plt.show()
```

Applications:

- Market Share
- Resource Allocation
- Attack Categories

Histogram

Displays data distribution.

```
import matplotlib.pyplot as plt

data = [10,20,20,30,40,40,50]

plt.hist(data)

plt.show()
```

Applications:

- Score Distribution
- Traffic Analysis
- Packet Size Analysis

Scatter Plot

Shows relationships between variables.

```
import matplotlib.pyplot as plt

x = [1,2,3,4,5]
y = [2,4,5,4,5]

plt.scatter(x,y)

plt.show()
```

Applications:

- Machine Learning
- Correlation Analysis
- Data Exploration

Area Plot

```
import matplotlib.pyplot as plt

x = [1,2,3,4]
y = [10,20,15,30]

plt.fill_between(x,y)

plt.show()
```

Applications:

- Revenue Analysis
- Resource Utilization

Grid Lines

```
plt.plot(x,y)

plt.grid(True)

plt.show()
```

Improves readability.

Legend

```
import matplotlib.pyplot as plt

x = [1,2,3]
```

```
plt.plot(  
    x,  
    [1,2,3],  
    label="Data A"  
)  
  
plt.plot(  
    x,  
    [3,2,1],  
    label="Data B"  
)  
  
plt.legend()  
  
plt.show()
```

Multiple Plots

```
import matplotlib.pyplot as plt  
  
x = [1,2,3]  
  
plt.plot(x,[1,2,3])  
  
plt.plot(x,[3,2,1])  
  
plt.show()
```

Figure Size

```
plt.figure(figsize=(8,5))  
  
plt.plot(x,y)  
  
plt.show()
```

Saving Graphs

```
plt.plot(x,y)  
  
plt.savefig("graph.png")
```

Supported formats:

```
PNG  
JPG  
PDF
```

SVG
EPS

Working with NumPy

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1,2,3,4])

y = np.array([10,20,30,40])

plt.plot(x,y)

plt.show()
```

Working with Pandas

```
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Marks": [85,90,78,92]
}

df = pd.DataFrame(data)

df.plot()

plt.show()
```

Subplots

Display multiple graphs in one figure.

```
import matplotlib.pyplot as plt

plt.subplot(1,2,1)
plt.plot([1,2,3])

plt.subplot(1,2,2)
plt.plot([3,2,1])

plt.show()
```

Real-World Example: Student Marks Analysis

```
import matplotlib.pyplot as plt

students = [
    "John",
    "Alice",
    "David",
    "Sam"
]

marks = [85,92,78,88]

plt.bar(
    students,
    marks
)

plt.title(
    "Student Performance"
)

plt.xlabel("Students")
plt.ylabel("Marks")

plt.show()
```

Matplotlib in Data Science

Used for:

- Exploratory Data Analysis (EDA)
- Trend Analysis
- Outlier Detection
- Statistical Visualization

Example:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("sales.csv")

df["Sales"].plot()

plt.show()
```

Matplotlib in Machine Learning

Used for:

- Training Loss Curves
- Accuracy Graphs
- Feature Analysis
- Model Evaluation

Example:

```
epochs = [1,2,3,4,5]

accuracy = [70,75,80,85,90]

plt.plot(
    epochs,
    accuracy
)

plt.show()
```

Matplotlib in Cyber Security**Failed Login Analysis**

```
import matplotlib.pyplot as plt

days = [
    "Mon",
    "Tue",
    "Wed",
    "Thu",
    "Fri"
]

attacks = [20,30,25,40,35]

plt.bar(days, attacks)

plt.title(
    "Failed Login Attempts"
)

plt.show()
```

Applications:

- Intrusion Detection
- Network Traffic Analysis

- Attack Pattern Analysis

Matplotlib in Digital Forensics

```
files = [  
    "Images",  
    "Documents",  
    "Videos"  
]  
  
count = [50,100,25]  
  
plt.pie(  
    count,  
    labels=files  
)  
  
plt.show()
```

Applications:

- Evidence Analysis
- Timeline Visualization
- Artifact Distribution

Common Plot Types

Plot Type	Purpose
plot()	Line Graph
bar()	Vertical Bar Chart
barh()	Horizontal Bar Chart
pie()	Pie Chart
hist()	Histogram
scatter()	Scatter Plot
fill_between()	Area Plot
boxplot()	Box Plot
stem()	Stem Plot

Advantages of Matplotlib

1. Easy to learn and use.
2. Produces publication-quality figures.
3. Highly customizable.
4. Integrates with NumPy and Pandas.
5. Supports multiple output formats.
6. Widely used in academia and industry.

Limitations of Matplotlib

1. More verbose than some modern visualization libraries.
2. Complex visualizations may require additional code.
3. Less aesthetically pleasing by default than libraries such as Seaborn.

Practical Experiment: Sales Analysis

```
import matplotlib.pyplot as plt

months = [
    "Jan",
    "Feb",
    "Mar",
    "Apr",
    "May"
]

sales = [
    100,
    150,
    200,
    180,
    250
]

plt.plot(
    months,
    sales,
    marker="o"
)

plt.title(
    "Monthly Sales"
)

plt.xlabel("Month")
```

```
plt.ylabel("Sales")

plt.grid(True)

plt.show()
```

Expected Result

A line graph showing sales growth from January to May.

Summary

Matplotlib is the foundational Python library for **data visualization**. It provides powerful tools for creating line charts, bar charts, pie charts, histograms, scatter plots, area charts, and many other graphical representations. Matplotlib integrates seamlessly with **NumPy**, **Pandas**, and Machine Learning libraries, making it an essential tool for Data Science, Artificial Intelligence, Business Analytics, Cyber Security, Digital Forensics, and Scientific Research. It enables users to transform raw data into meaningful visual insights that support analysis, decision-making, and communication.

4. Seaborn

Introduction

Seaborn is a high-level Python data visualization library built on top of **Matplotlib**. It provides an attractive and informative interface for creating statistical graphics with minimal code.

Seaborn is widely used in:

- Data Science
- Machine Learning
- Artificial Intelligence
- Business Intelligence
- Statistical Analysis
- Research Analytics
- Cyber Security Analytics
- Financial Data Analysis

What is Seaborn?

Seaborn simplifies the process of creating beautiful and meaningful visualizations. While Matplotlib provides the foundation, Seaborn adds:

- Better default styles
- Statistical plotting functions
- Attractive color palettes
- Integration with Pandas DataFrames
- Advanced visualization capabilities

Why Use Seaborn?

Using Matplotlib

```
import matplotlib.pyplot as plt

marks = [85, 90, 78, 92]

plt.plot(marks)

plt.show()
```

Using Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

marks = [85, 90, 78, 92]

sns.lineplot(x=[1,2,3,4], y=marks)

plt.show()
```

Seaborn automatically provides:

- Better formatting
- Improved color schemes
- Cleaner visualization

Features of Seaborn

1. Built on Matplotlib.
2. Beautiful default themes.
3. Statistical visualization support.
4. Easy integration with Pandas.
5. Advanced plotting functions.

6. Correlation and regression analysis.
7. Distribution visualization.
8. Heatmap generation.

Installation

Install Seaborn using PIP:

```
pip install seaborn
```

Verify installation:

```
import seaborn as sns  
  
print(sns.__version__)
```

Importing Seaborn

```
import seaborn as sns  
import matplotlib.pyplot as plt
```

Built-in Datasets

Seaborn provides sample datasets.

View Available Datasets

```
import seaborn as sns  
  
print(sns.get_dataset_names())
```

Load Dataset

```
import seaborn as sns  
  
tips = sns.load_dataset("tips")  
  
print(tips.head())
```

Output:

```
total bill  tip  sex  
0    16.99  1.01 Female  
1    10.34  1.66  Male  
...
```

Seaborn Themes

Seaborn provides attractive themes.

Default Theme

```
sns.set_theme()
```

Available Themes

```
darkgrid  
whitegrid  
dark  
white  
ticks
```

Example:

```
sns.set_style("darkgrid")
```

Line Plot

Used to show trends over time.

```
import seaborn as sns  
import pandas as pd  
import matplotlib.pyplot as plt  
  
data = pd.DataFrame({  
    "Month": [1, 2, 3, 4, 5],  
    "Sales": [100, 150, 200, 180, 250]  
})  
  
sns.lineplot(  
    x="Month",  
    y="Sales",  
    data=data  
)  
  
plt.show()
```

Applications:

- Sales Trends
- Stock Prices
- Website Traffic
- Network Monitoring

Bar Plot

Used to compare categories.

```
import seaborn as sns
import pandas as pd

data = pd.DataFrame({
    "Student":
        ["John","Alice","David"],
    "Marks":
        [85,92,78]
})

sns.barplot(
    x="Student",
    y="Marks",
    data=data
)

plt.show()
```

Count Plot

Displays frequency counts.

```
import seaborn as sns

tips = sns.load_dataset("tips")

sns.countplot(
    x="day",
    data=tips
)

plt.show()
```

Applications:

- Customer Analysis
- Log Frequency Analysis
- Attack Count Analysis

Histogram

Shows data distribution.

```
import seaborn as sns

tips = sns.load_dataset("tips")
```

```
sns.histplot(  
    tips["total_bill"]  
)  
  
plt.show()
```

Applications:

- Exam Score Analysis
- Packet Size Analysis
- Salary Distribution

KDE Plot (Kernel Density Estimation)

Shows probability density.

```
sns.kdeplot(  
    tips["total_bill"]  
)  
  
plt.show()
```

Produces a smooth distribution curve.

Scatter Plot

Shows relationships between variables.

```
sns.scatterplot(  
    x="total_bill",  
    y="tip",  
    data=tips  
)  
  
plt.show()
```

Applications:

- Correlation Analysis
- Machine Learning
- Feature Exploration

Regression Plot

Displays scatter plot with regression line.

```
sns.regplot(  
    x="total_bill",  
    y="tip",  
    data=tips  
)  
  
plt.show()
```

Applications:

- Predictive Analysis
- Trend Identification
- Statistical Modeling

Box Plot

Displays distribution and outliers.

```
sns.boxplot(  
    y="total_bill",  
    data=tips  
)  
  
plt.show()
```

Box Plot Components

```
Maximum  
|  
Q3 — Box Top  
Median  
Q1 — Box Bottom  
|  
Minimum
```

Applications:

- Outlier Detection
- Fraud Analysis
- Anomaly Detection

Violin Plot

Combination of Box Plot and KDE Plot.

```
sns.violinplot(  
    y="total_bill",  
    data=tips  
)
```

```
)  
plt.show()
```

Shows:

- Distribution
- Density
- Median

Pair Plot

Displays relationships among all numerical variables.

```
sns.pairplot(tips)  
plt.show()
```

Useful for:

- Exploratory Data Analysis (EDA)
- Machine Learning

Heatmap

One of the most powerful Seaborn visualizations.

Correlation Heatmap

```
import seaborn as sns  
import pandas as pd  
  
data = {  
    "Math": [80, 85, 90, 95],  
    "Science": [78, 88, 89, 97],  
    "English": [75, 82, 91, 93]  
}  
  
df = pd.DataFrame(data)  
  
corr = df.corr()  
  
sns.heatmap(  
    corr,  
    annot=True  
)  
  
plt.show()
```

Example Output:

	Math	Science	English
Math	1.00	0.95	0.92
Science	0.95	1.00	0.89
English	0.92	0.89	1.00

Applications:

- Feature Selection
- Data Analysis
- Security Event Correlation

Heatmap in Cyber Security

```
import seaborn as sns
import pandas as pd

attacks = pd.DataFrame([
    [50,30,20],
    [40,60,25],
    [35,45,70]
])

sns.heatmap(
    attacks,
    annot=True
)

plt.show()
```

Applications:

- Threat Intelligence
- Attack Correlation
- Network Analysis

Distribution Plot

```
sns.displot(
    tips["total_bill"]
)

plt.show()
```

Displays:

- Histogram

- Density Curve

Joint Plot

Shows relationship and distributions together.

```
sns.jointplot(  
    x="total_bill",  
    y="tip",  
    data=tips  
)  
  
plt.show()
```

Applications:

- Statistical Analysis
- Feature Relationships

Categorical Plot

```
sns.catplot(  
    x="day",  
    y="total_bill",  
    kind="bar",  
    data=tips  
)  
  
plt.show()
```

Working with Pandas DataFrame

```
import pandas as pd  
import seaborn as sns  
  
data = {  
    "Student":  
        ["John", "Alice", "David"],  
    "Marks":  
        [85, 92, 78]  
}  
  
df = pd.DataFrame(data)  
  
sns.barplot(  
    x="Student",  
    y="Marks",
```

```
data=df  
)
```

Color Palettes

Seaborn provides attractive palettes.

```
sns.color_palette()
```

Popular palettes:

```
deep  
muted  
bright  
pastel  
dark  
colorblind
```

Example:

```
sns.set_palette("pastel")
```

Statistical Analysis Using Seaborn

```
import seaborn as sns  
  
tips = sns.load_dataset("tips")  
  
sns.boxplot(  
    x="day",  
    y="total_bill",  
    data=tips  
)
```

Helps identify:

- Median
- Quartiles
- Outliers

Seaborn in Data Science

Applications:

- Exploratory Data Analysis
- Data Cleaning Validation
- Feature Correlation

- Trend Analysis

Example:

```
df.corr()

sns.heatmap(
    df.corr(),
    annot=True
)
```

Seaborn in Machine Learning

Applications:

- Feature Selection
- Correlation Analysis
- Model Evaluation
- Data Exploration

Example:

```
sns.pairplot(dataset)
```

Seaborn in Artificial Intelligence

Applications:

- Dataset Visualization
- Neural Network Performance
- Feature Importance Analysis

Example:

```
sns.lineplot(
    x=epochs,
    y=accuracy
)
```

Seaborn in Cyber Security

Applications:

- Intrusion Detection Analysis
- Log Pattern Visualization

- Network Traffic Analysis
- Threat Intelligence

Example:

```
sns.countplot(
    x="AttackType",
    data=attack_logs
)
```

Important Seaborn Functions

Function	Purpose
lineplot()	Line Graph
barplot()	Bar Chart
countplot()	Frequency Count
histplot()	Histogram
scatterplot()	Scatter Plot
regplot()	Regression Analysis
boxplot()	Outlier Detection
violinplot()	Distribution Analysis
heatmap()	Correlation Matrix
pairplot()	Multi-variable Analysis
displot()	Distribution Plot
jointplot()	Relationship Analysis
catplot()	Categorical Analysis

Matplotlib vs Seaborn

Feature	Matplotlib	Seaborn
Complexity	More Code	Less Code
Appearance	Basic	Attractive
Statistical Plots	Limited	Extensive
Integration with Pandas	Good	Excellent
Learning Curve	Moderate	Easy

Advantages of Seaborn

1. Beautiful default styles.
2. Less code than Matplotlib.
3. Excellent statistical visualizations.
4. Strong Pandas integration.
5. Advanced plotting functions.
6. Easy correlation analysis.

Limitations of Seaborn

1. Built on Matplotlib, so some customization still requires Matplotlib knowledge.
2. Less flexible than pure Matplotlib for highly customized plots.
3. Slightly slower for extremely large datasets.

Practical Experiment: Student Marks Analysis

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

data = {
    "Student":
        ["John","Alice","David","Sam"],
    "Marks":
        [85,92,78,88]
}

df = pd.DataFrame(data)

sns.barplot(
    x="Student",
    y="Marks",
    data=df
)

plt.title("Student Performance")

plt.show()
```

Expected Result

A professional-looking bar chart displaying marks obtained by each student.

Summary

Seaborn is a high-level statistical data visualization library built on top of Matplotlib. It simplifies the creation of attractive and informative visualizations such as line plots, bar charts, histograms, scatter plots, box plots, violin plots, pair plots, and heatmaps. Seaborn integrates seamlessly with **Pandas** and **NumPy**, making it one of the most important tools for Data Science, Machine Learning, Artificial Intelligence, Cyber Security Analytics, Business Intelligence, and Research Computing. It is especially valuable for Exploratory Data Analysis (EDA), correlation analysis, and statistical visualization.

5. Scikit-Learn

Introduction

Scikit-Learn (sklearn) is one of the most popular open-source Python libraries for **Machine Learning (ML)**. It provides simple and efficient tools for data mining, data analysis, predictive modeling, and machine learning.

Scikit-Learn is built on top of:

- NumPy
- SciPy
- Matplotlib
- Pandas

It is widely used in:

- Data Science
- Artificial Intelligence
- Predictive Analytics
- Cyber Security Analytics
- Financial Forecasting
- Healthcare Analytics
- Fraud Detection
- Research and Education

What is Machine Learning?

Machine Learning is a branch of AI that enables computers to learn patterns from data and make predictions without being explicitly programmed.

Traditional Programming

Data + Rules



Program



Output

Machine Learning

Data + Output



Learning Algorithm



Model



Predictions

Why Scikit-Learn?

Without Scikit-Learn:

- Complex mathematical implementations
- Manual algorithm development
- More coding effort

With Scikit-Learn:

```
from sklearn.linear_model import LinearRegression
```

A complete machine learning algorithm becomes available with a single import.

Features of Scikit-Learn

1. Easy to learn and use.
2. Large collection of ML algorithms.
3. Data preprocessing tools.
4. Feature selection methods.
5. Model evaluation tools.
6. Cross-validation support.
7. Hyperparameter tuning.
8. Integration with NumPy and Pandas.

Installation

Install Scikit-Learn using PIP:

```
pip install scikit-learn
```

Verify installation:

```
import sklearn  
  
print(sklearn.__version__)
```

Machine Learning Workflow

```
graph TD  
    A[Collect Data] --> B[Preprocess Data]  
    B --> C[Split Dataset]  
    C --> D[Train Model]  
    D --> E[Test Model]  
    E --> F[Evaluate Performance]  
    F --> G[Deploy Model]
```

Importing Scikit-Learn

```
import sklearn
```

Usually specific modules are imported:

```
from sklearn.linear_model import LinearRegression
```

Types of Machine Learning

1. Supervised Learning

Uses labeled data.

Examples:

- House Price Prediction
- Student Grade Prediction
- Spam Detection

Algorithms:

- Linear Regression

- Logistic Regression
- Decision Tree
- Random Forest
- Support Vector Machine

2. Unsupervised Learning

Uses unlabeled data.

Examples:

- Customer Segmentation
- Network Traffic Clustering

Algorithms:

- K-Means
- DBSCAN
- Hierarchical Clustering

3. Reinforcement Learning

Learning through rewards and penalties.

Examples:

- Robotics
- Self-Driving Cars
- Game Playing

(Note: Reinforcement Learning is not the primary focus of Scikit-Learn.)

Data Representation

Machine learning data is usually stored as:

X = Features
y = Target

Example:

Study Hours	Marks
2	40

4	60
6	80

```
X = [[2],[4],[6]]
y = [40,60,80]
```

Dataset Splitting

Training and testing datasets are created using:

```
from sklearn.model_selection import train_test_split
```

Example:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2
)
```

Linear Regression

Definition

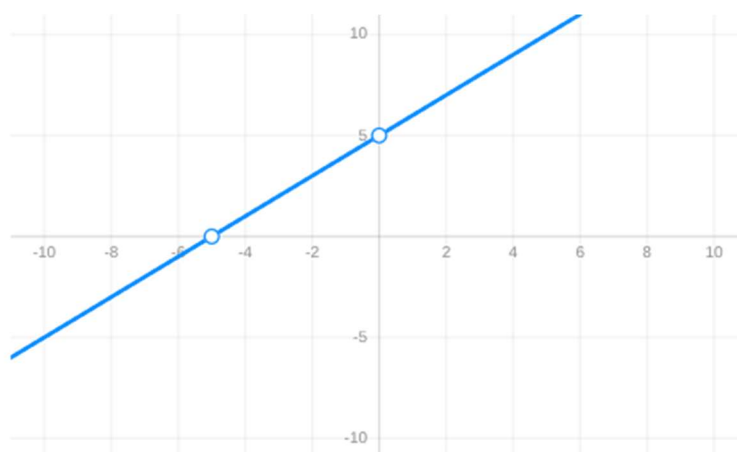
Linear Regression predicts continuous values using a straight-line relationship.

Formula

$$y = mx + b$$

$$m = 1.0$$

$$b = 5.0$$



Where:

- y = Predicted value
- m = Slope
- x = Input variable
- b = Intercept

Linear Regression Example

```
from sklearn.linear_model import LinearRegression

X = [[2],[4],[6],[8]]
y = [40,60,80,100]

model = LinearRegression()

model.fit(X, y)

prediction = model.predict([[10]])

print(prediction)
```

Output:

```
[120.]
```

Logistic Regression

Used for classification problems.

Example:

```
from sklearn.linear_model import LogisticRegression
```

Applications:

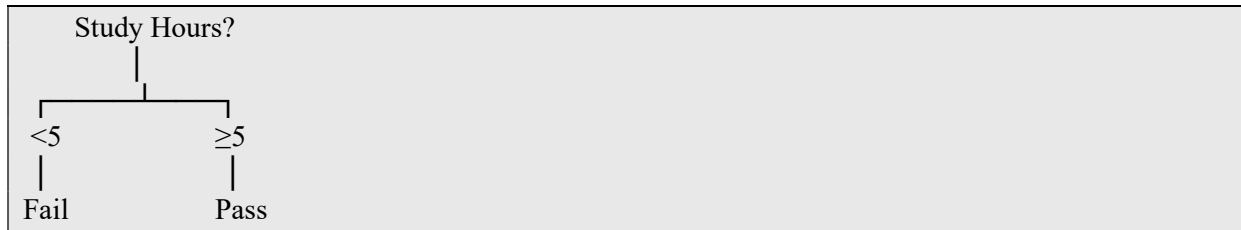
- Spam Detection
- Fraud Detection
- Disease Prediction

Possible Outputs:

```
Yes / No
Spam / Not Spam
Fraud / Legitimate
```

Decision Trees

Decision trees make predictions using a tree-like structure.



Example:

```
from sklearn.tree import DecisionTreeClassifier
```

Applications:

- Credit Approval
- Medical Diagnosis
- Security Risk Assessment

Random Forest

Random Forest combines multiple decision trees.

```
from sklearn.ensemble import RandomForestClassifier
```

Advantages:

- High accuracy
- Reduced overfitting
- Robust predictions

Applications:

- Intrusion Detection
- Fraud Detection
- Customer Analytics

Support Vector Machine (SVM)

Used for classification and regression.

```
from sklearn.svm import SVC
```

Applications:

- Face Recognition

- Malware Detection
- Text Classification

K-Nearest Neighbors (KNN)

Classifies data based on nearby data points.

```
from sklearn.neighbors import KNeighborsClassifier
```

Example:

```
New Student
|
Nearest Students
|
Majority Class
|
Prediction
```

Naive Bayes

Based on probability theory.

```
from sklearn.naive_bayes import GaussianNB
```

Applications:

- Spam Filtering
- Text Classification
- Sentiment Analysis

Clustering with K-Means

K-Means is an unsupervised learning algorithm.

```
from sklearn.cluster import KMeans
```

Example:

```
from sklearn.cluster import KMeans

X = [
    [1,2],
    [2,3],
    [8,8],
    [9,9]
]
```

```
model = KMeans(n_clusters=2)

model.fit(X)

print(model.labels_)
```

Output:

```
[0 0 1 1]
```

Data Preprocessing

Machine learning requires clean data.

Standardization

```
from sklearn.preprocessing import StandardScaler
```

Example:

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaled_data = scaler.fit_transform(X)
```

Normalization

```
from sklearn.preprocessing import MinMaxScaler
```

Example:

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()

normalized_data = scaler.fit_transform(X)
```

Encoding Categorical Data

```
from sklearn.preprocessing import LabelEncoder
```

Example:

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
```

```
gender = [
    "Male",
    "Female",
    "Male"
]

encoded = encoder.fit_transform(gender)

print(encoded)
```

Output:

```
[1 0 1]
```

Model Evaluation

Accuracy Score

```
from sklearn.metrics import accuracy_score
```

Example:

```
accuracy_score(
    y_true,
    y_pred
)
```

Output:

```
0.95
```

Meaning:

```
95% Accurate
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
```

Example:

```
print(
    confusion_matrix(
        y_true,
        y_pred
    )
)
```

Output:

```
[[50 5]
 [3 42]]
```

Classification Report

from sklearn.metrics import classification_report

Provides:

- Precision
- Recall
- F1-Score
- Accuracy

Cross Validation

Used to validate model performance.

from sklearn.model_selection import cross_val_score

Example:

```
scores = cross_val_score(
    model,
    X,
    y,
    cv=5
)
```

Hyperparameter Tuning

Used to improve model performance.

```
from sklearn.model_selection import GridSearchCV
```

Example:

```
grid = GridSearchCV(
    model,
    parameters
)
```

Pipeline

Automates preprocessing and training.

```
from sklearn.pipeline import Pipeline
```

Example:

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", LogisticRegression())
])
```

Working with Real Datasets

Iris Dataset

Provided by Scikit-Learn.

```
from sklearn.datasets import load_iris

iris = load_iris()

print(iris.data.shape)
```

Output:

```
(150, 4)
```

Meaning:

```
150 Samples
4 Features
```

Complete Machine Learning Example

Student Marks Prediction

```
from sklearn.linear_model import LinearRegression

X = [[2],[4],[6],[8]]
y = [40,60,80,100]

model = LinearRegression()

model.fit(X,y)

prediction = model.predict([[10]])

print(prediction)
```

Output:

```
[120.]
```

Scikit-Learn in Data Science

Applications:

- Predictive Analytics
- Customer Behavior Analysis
- Sales Forecasting
- Market Research

Scikit-Learn in Machine Learning

Applications:

- Classification
- Regression
- Clustering
- Recommendation Systems

Scikit-Learn in AI

Applications:

- Intelligent Systems
- Decision Support Systems
- Pattern Recognition

Scikit-Learn in Cyber Security

Applications:

Malware Detection

```
from sklearn.ensemble import RandomForestClassifier
```

Intrusion Detection

```
from sklearn.svm import SVC
```

Spam Email Classification

```
from sklearn.naive_bayes import GaussianNB
```

Scikit-Learn in Digital Forensics

Applications:

- Evidence Classification
- Log Analysis
- User Behavior Profiling
- Anomaly Detection

Common Scikit-Learn Modules

Module	Purpose
sklearn.linear_model	Regression Models
sklearn.tree	Decision Trees
sklearn.ensemble	Random Forest
sklearn.svm	Support Vector Machines
sklearn.cluster	Clustering
sklearn.metrics	Evaluation
sklearn.preprocessing	Data Preparation
sklearn.model_selection	Dataset Splitting
sklearn.pipeline	Workflow Automation
sklearn.datasets	Sample Datasets

Advantages of Scikit-Learn

1. Easy to use.
2. Large collection of algorithms.
3. Excellent documentation.
4. Strong community support.
5. Integrates with NumPy and Pandas.
6. Suitable for beginners and professionals.

Limitations of Scikit-Learn

1. Limited deep learning support.
2. Not ideal for very large distributed datasets.

3. Less suitable than specialized frameworks such as TensorFlow or PyTorch for complex neural networks.

Practical Experiment: Iris Flower Classification

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2
)

model = DecisionTreeClassifier()

model.fit(X_train, y_train)

predictions = model.predict(X_test)

print(
    "Accuracy:",
    accuracy_score(
        y_test,
        predictions
    )
)
```

Expected Output

```
Accuracy: 0.95
```

(Actual value may vary.)

Summary

Scikit-Learn (sklearn) is the most widely used Python library for classical Machine Learning. It provides powerful tools for data preprocessing, regression, classification, clustering, model evaluation, cross-validation, and hyperparameter tuning. With support for algorithms such as Linear Regression, Logistic Regression, Decision Trees, Random Forests, SVM, KNN, Naive Bayes, and K-Means, Scikit-Learn forms the backbone of many Data Science, AI, Cyber Security, and Predictive Analytics applications. It is often the first machine learning library learned by students and professionals before moving to advanced deep learning frameworks.

6. TensorFlow

Purpose

Deep Learning and AI

Installation

```
pip install tensorflow
```

Example

```
import tensorflow as tf  
  
print(tf.__version__)
```

Applications:

- Neural Networks
- Computer Vision
- NLP

7. PyTorch

Purpose

Deep Learning

Installation

```
pip install torch
```

Applications:

- AI Research
- Image Recognition
- Natural Language Processing

8. OpenCV

Purpose

Computer Vision

Installation

```
pip install opencv-python
```

Example

```
import cv2  
  
image = cv2.imread("image.jpg")
```

Applications:

- Face Detection
- Image Processing
- Video Analytics

9. Requests

Purpose

HTTP Communication

Installation

```
pip install requests
```

Example

```
import requests  
  
response = requests.get(  
    "https://example.com"  
)  
  
print(response.status_code)
```

Applications:

- API Integration
- Web Scraping
- Automation

10. BeautifulSoup

Purpose

Web Scraping

Installation

```
pip install beautifulsoup4
```

Example

```
from bs4 import BeautifulSoup
```

Applications:

- Website Data Extraction
- Market Research
- Data Collection

Cyber Security Libraries

Library	Purpose
Scapy	Packet Analysis
Nmap	Network Scanning
Paramiko	SSH Automation
Hashlib	Hash Generation
Cryptography	Encryption
Socket	Network Programming

Example

```
import hashlib

text = "Python"

hash_value = hashlib.sha256(
    text.encode()
).hexdigest()

print(hash_value)
```

Data Science Libraries

Library	Purpose
NumPy	Numerical Computing
Pandas	Data Analysis
Matplotlib	Visualization
Seaborn	Statistical Visualization
SciPy	Scientific Computing

AI and Machine Learning Libraries

Library	Purpose
TensorFlow	Deep Learning
PyTorch	Deep Learning
Scikit-Learn	Machine Learning
Keras	Neural Networks
Transformers	Large Language Models

Web Development Libraries

Library	Purpose
Flask	Lightweight Web Apps
Django	Enterprise Web Apps
FastAPI	APIs
Requests	HTTP Communication

Digital Forensics Libraries

Library	Purpose
Pytsk3	Disk Forensics
Volatility	Memory Forensics
Hashlib	File Integrity
OS	File System Analysis

Practical Example: Data Analysis

```
import pandas as pd

data = {
    "Marks": [85, 90, 78, 88]
}

df = pd.DataFrame(data)

print(df["Marks"].mean())
```

Output:

```
85.25
```

Practical Example: Visualization

```
import matplotlib.pyplot as plt

marks = [85, 90, 78, 88]
```

```
plt.bar(  
    ["A","B","C","D"],  
    marks  
)  
  
plt.show()
```

Advantages of Python Libraries

- Reduce development time.
- Provide tested and optimized code.
- Improve productivity.
- Enable advanced functionality.
- Support rapid application development.
- Facilitate scientific and AI applications.

Best Practices

Import only required libraries.

Use aliases where appropriate.

```
import numpy as np  
import pandas as pd
```

Keep libraries updated.

Use virtual environments.

Read official documentation before use.

Summary

Python Libraries are collections of pre-written modules, functions, classes, and packages that provide reusable functionality for software development. Python offers both standard libraries (such as math, os, json, and datetime) and third-party libraries (such as NumPy, Pandas, Matplotlib, TensorFlow, and Scikit-Learn). These libraries make Python one of the most powerful programming languages for web development, data science, artificial intelligence, machine learning, cyber security, digital forensics, automation, and scientific computing.